TESTLEAF TRAINING

INTERVIEW PREPARATION – WEEK 1

JavaScript Fundamentals

How to Use This Document

Use this document to prepare for JavaScript interviews. Read each question, try answering it aloud, then compare your response with the answer and examples. Output-based questions are useful for practising code tracing. The material is organized into JavaScript data types, operators, and conditional statements.

Contents

- Part 1 – JavaScript Data Types
- Part 2 – JavaScript Operators and Conditional Statements

PART 1 – JAVASCRIPT DATA TYPES

Interview questions and answers covering primitive data types, number precision, BigInt, Symbol, scenario-based questions, output-based questions, and quick revision.

These interview questions are based on the JavaScript setup and data type examples covered in `datatypes.js`. They are suitable for beginners and learners preparing for JavaScript or Playwright automation interviews.

1. Basic Interview Questions

Question 1. What is a data type in JavaScript?

Answer: A data type describes the kind of value a variable holds, such as a number, string, boolean, undefined, null, BigInt, or Symbol.

Example:

```
// Store a number in a variable
var mark = -99;

// Display the type of the variable
console.log(typeof mark);
```

Output:

```
number
```

Question 2. What are the primitive data types in JavaScript?

Answer: JavaScript has seven primitive data types:

1. `number`
2. `string`
3. `boolean`
4. `undefined`
5. `null`
6. `bigint`
7. `symbol`

Example:

```
// Store a string value
var courseName = "Playwright";

// Store a boolean value
var isEnabled = true;

// Display their data types
console.log(typeof courseName);
console.log(typeof isEnabled);
```

Output:

```
string
boolean
```

Question 3. What is the difference between `console.log()` and `typeof`?

Answer:

- `console.log()` displays a value or message.
- `typeof` returns a string describing the type of a value.

Example:

```
// Store a number
var mark = -99;

// Display the value
console.log(mark);

// Display the data type
console.log(typeof mark);
```

Output:

```
-99
number
```

Question 4. What is the `number` data type in JavaScript?

Answer: The `number` type represents integers and floating-point numbers. It also includes special numeric values such as `NaN` and `Infinity`.

Example:

```
// Store an integer
var mark = -99;

// Store a decimal number
var rateOfInterest = 10 / 3;

// Display both values
console.log(mark);
console.log(rateOfInterest);
```

Question 5. What is a string in JavaScript?

Answer: A string is a sequence of text characters. Strings can be written using single quotes, double quotes, or backticks.

Example:

```
// Create a string using backticks
var courseName = `Playwright`;

// Display the value and data type
console.log(courseName);
console.log(typeof courseName);
```

Output:

```
Playwright
string
```

Question 6. What is a boolean data type?

Answer: A boolean represents one of two values: `true` or `false`. It is commonly used in conditions and to represent states.

Example:

```
// Store whether a feature is enabled
var isEnabled = true;

// Display the value and its type
console.log(isEnabled);
console.log(typeof isEnabled);
```

Output:

```
true
boolean
```

Question 7. What is `undefined` in JavaScript?

Answer: A variable declared without an assigned value has the value `undefined`.

Example:

```
// Declare a variable without assigning a value
var emailID;

// Display the value and its type
console.log(emailID);
console.log(typeof emailID);
```

Output:

```
undefined
undefined
```

Question 8. What is `null` in JavaScript?

Answer: `null` is a value that is intentionally assigned to represent the absence of a value.

Example:

```
// Intentionally represent the absence of a middle name
var middleName = null;

// Display the value
console.log(middleName);
```

Output:

```
null
```

Question 9. What is the difference between `undefined` and `null`?

Answer:

- `undefined` commonly means a value has not been assigned.
- `null` is intentionally assigned to represent no value.

Example:

```
// Declare a variable without assigning a value
var emailID;

// Intentionally assign no value to another variable
var middleName = null;

// Display both values
console.log(emailID);
console.log(middleName);
```

Output:

```
undefined
null
```

Question 10. Why does `typeof null` return `"object"`?

Answer: This is a long-standing historical quirk in JavaScript. Although `typeof null` returns `"object"`, `null` is a primitive value, not an object.

Example:

```
// Assign null to a variable
```

```
var middleName = null;

// Check its type
console.log(typeof middleName);
```

Output:

```
object
```

2. Number Precision and BigInt Questions

Question 11. What is `Number.MAX_SAFE_INTEGER`?

Answer: `Number.MAX_SAFE_INTEGER` is the largest integer that JavaScript's `number` type can represent safely without losing integer precision. Its value is `9007199254740991`.

Example:

```
// Read the maximum safe integer
var maxNumber = Number.MAX_SAFE_INTEGER;

// Display the value
console.log(maxNumber);
```

Output:

```
9007199254740991
```

Question 12. What does “safe integer” mean in JavaScript?

Answer: A safe integer is an integer that JavaScript can represent exactly and compare correctly using the `number` type. The safe integer range is from `-(2^53 - 1)` to `2^53 - 1`.

Question 13. What is BigInt in JavaScript?

Answer: `BigInt` is a primitive data type used to represent integers of arbitrary size, subject to available memory. An integer literal can be written with an `n` suffix.

Example:

```
// Store a large integer as BigInt
var num2 = 9007199254740995n;

// Display its value and data type
console.log(num2);
console.log(typeof num2);
```

Output:

```
9007199254740995n
bigint
```

Question 14. What is the difference between `number` and `bigint`?

Answer:

`number`	`bigint`
Represents integers and decimal numbers	Represents integers only
Integers are safe only within the safe integer range	Can represent much larger integers exactly
Example: `100`	Example: `100n`
`typeof` returns `"number"`	`typeof` returns `"bigint"`

Question 15. Why do we add `n` at the end of a number?

Answer: The `n` suffix identifies a BigInt literal. Without the suffix, an integer literal is normally interpreted as a `number`.

Example:

```
// Create a number value
var regularNumber = 100;

// Create a BigInt value
var bigIntNumber = 100n;

// Display the data types
console.log(typeof regularNumber);
console.log(typeof bigIntNumber);
```

Output:

```
number
bigint
```

Question 16. Can we perform arithmetic directly between a number and a BigInt?

Answer: No. JavaScript does not allow most arithmetic operations between `number` and `bigint` values without explicit conversion.

Example:

```
// Create a number
var num1 = 10;

// Create a BigInt
var num2 = 10n;

// This operation throws a TypeError
// console.log(num1 + num2);
```

Explanation: Convert values deliberately when appropriate, keeping in mind that converting a large BigInt to a number can lose precision.

Question 17. Why might a very large integer stored as a number lose precision?

Answer: JavaScript's `number` type uses IEEE 754 double-precision floating-point representation. Integers outside the safe integer range may not all be represented exactly. Use BigInt when exact large-integer arithmetic is needed.

3. Symbol Questions

Question 18. What is a Symbol in JavaScript?

Answer: A Symbol is a primitive data type that creates a unique value. Symbols are often used as unique property keys.

Example:

```
// Create a Symbol with a description
var userID = Symbol("id");

// Display the Symbol
console.log(userID);

// Display its data type
console.log(typeof userID);
```

Output:

```
Symbol(id)
symbol
```

Question 19. If two Symbols have the same description, are they equal?

Answer: No. Each call to `Symbol()` creates a unique Symbol.

Example:

```
// Create two Symbols with the same description
var firstID = Symbol("id");
var secondID = Symbol("id");

// Compare the two Symbols
console.log(firstID === secondID);
```

Output:

```
false
```

4. Scenario-Based Interview Questions

Question 20. You declare a variable for a learner's email, but you do not know the email yet. What value does the variable contain?

Answer: If the variable is declared without assigning a value, its value is `undefined`.

```
// Declare the email variable without a value
var emailID;

// Check its value
console.log(emailID);
```

Expected output: `undefined`

Question 21. An API response contains no middle name for a user. You want to represent that absence intentionally. Which value could you use?

Answer: Use `null` if that matches the API or application contract.

```
// Represent the missing middle name explicitly
var middleName = null;

// Check the value
console.log(middleName);
```

Expected output: `null`

Follow-up: Why might `typeof middleName` be confusing?

Answer: It returns `"object"` because of JavaScript's historical `typeof null` behavior.

Question 22. You run `console.log(typeof num3)` and get `"bigint"`. What does this tell you?

Answer: It tells you that `num3` contains a BigInt value.

```
// Store a large integer as BigInt
var num3 = 9007199254740997n;

// Display its type
console.log(typeof num3);
```

Expected output: `bigint`

Question 23. You need to store a very large integer identifier and preserve its exact integer value. What should you consider?

Answer: Consider using BigInt if the value is handled as an integer within JavaScript. For JSON APIs, remember that `JSON.stringify()` does not serialize BigInt by default; the API may instead represent large identifiers as strings.

Question 24. A learner says that `typeof middleName` proves `null` is an object. Is the learner correct?

Answer: No. `typeof null` returns `"object"` because of a historical quirk. `null` itself is a primitive value.

Question 25. You want to check whether a variable contains a boolean value. Which expression can you use?

Answer: Use `typeof variableName === "boolean"`.

```
// Store a boolean value
var isEnabled = true;

// Check whether the variable contains a boolean
console.log(typeof isEnabled === "boolean");
```

Expected output: `true`

5. Output-Based Interview Questions

Try to predict each output before checking the answer.

Question 26. What is the output?

```
// Store a negative number
var mark = -99;

// Print its type
console.log(typeof mark);
```

Answer:

```
number
```

Question 27. What is the output?

```
// Declare a variable without assigning a value
var emailID;

// Print its type
console.log(typeof emailID);
```

Answer:

```
undefined
```

Question 28. What is the output?

```
// Assign null to a variable
var middleName = null;

// Print its type
console.log(typeof middleName);
```

Answer:

```
object
```

Question 29. What is the output?

```
// Store an integer as BigInt
var num3 = 9007199254740997n;
```

```
// Print the value
console.log(num3);

// Print the type
console.log(typeof num3);
```

Answer:

```
9007199254740997n
bigint
```

Question 30. What is the output?

```
// Store a number
var regularNumber = 100;

// Store a BigInt
var bigIntNumber = 100n;

// Compare their types
console.log(typeof regularNumber === typeof bigIntNumber);
```

Answer:

```
false
```

Explanation: The first type is `"number"` and the second is `"bigint"`.

6. Quick Revision Table

Concept	Key point
`console.log(value)`	Displays a value
`typeof value`	Returns a string describing the value's type
`number`	Represents integers and decimal numbers
`string`	Represents text
`boolean`	Represents `true` or `false`
`undefined`	A declared variable has no assigned value
`null`	Intentionally represents the absence of a value
`typeof null`	Returns `"object"` due to a historical quirk
`Number.MAX_SAFE_INTEGER`	`9007199254740991`
`bigint`	Represents large integers exactly
`n` suffix	Identifies a BigInt integer literal
`symbol`	Creates a unique primitive value

7. Practice Questions for Learners

Answer these without referring to the notes:

8. What is the difference between `console.log()` and `typeof`?
9. Name the seven primitive data types in JavaScript.
10. What is the difference between `undefined` and `null`?
11. Why does `typeof null` return `"object"`?
12. What is the value of `Number.MAX_SAFE_INTEGER`?
13. Why would you use BigInt?
14. How can you identify a BigInt literal?
15. Can you directly add a `number` and a `bigint`?
16. What is the purpose of a Symbol?

17. What is the output of `console.log(typeof true)`?

Answer to Question 10

```
boolean
```

PART 2 – JAVASCRIPT OPERATORS AND CONDITIONAL STATEMENTS

Interview questions and answers covering assignment, arithmetic, comparison, equality, increment/decrement, logical operators, conditional statements, and practical scenarios.

Assignment and Arithmetic Operators

Question 31. What is the assignment operator in JavaScript?

The assignment operator (`=`) assigns a value to a variable. For example, `var age = 10;` stores `10` in `age`.

Question 32. What are arithmetic operators?

They perform mathematical operations such as addition, subtraction, multiplication, division, exponentiation, and finding a remainder.

Question 33. What is the output of `10 + 3`, `10 - 3`, and `10 * 3`?

The outputs are `13`, `7`, and `30`, respectively.

Question 34. What does the exponentiation operator (`**`) do?

It raises the left operand to the power of the right operand. For example, `10 ** 3` returns `1000`.

Question 35. What is the difference between `/` and `%`?

`/` performs division. `%` returns the remainder. For example, `10 / 3` is approximately `3.3333333333333335`, while `10 % 3` is `1`.

Question 36. What does `Math.floor()` do?

It rounds a number down to the nearest integer. For example, `Math.floor(10 / 3)` returns `3`.

Comparison and Equality Operators

Question 37. What do comparison operators return?

They return a Boolean value, either `true` or `false`.

Question 38. What is the difference between `<` and `<=`?

`<` checks whether one value is strictly less than another. `<=` also returns true when the values are equal.

Question 39. What is the difference between `==` and `===`?

`==` compares values and may perform type conversion. `===` compares both value and data type without type conversion.

Question 40. What is the output of `10 == "10"`?

It returns `true` because `==` converts the string `"10"` to a number for this comparison.

Question 41. What is the output of `10 === "10"`?

It returns `false` because `10` is a number and `"10"` is a string.

Question 42. What is the output of the following code?

```
var x = 10;
var y = 20;
console.log(x < y);
console.log(x > y);
```

Output:

```
true
false
```

Increment and Decrement Operators

Question 43. What is the difference between `v++` and `++v`?

`v++` returns the current value and then increments it. `++v` increments the value first and then returns the updated value.

Question 44. What is the output of this code?

```
var v = 5;
console.log(v++);
console.log(v);
```

Output:

```
5
6
```

Question 45. What is the difference between `v--` and `--v`?

`v--` returns the current value and then decrements it. `--v` decrements the value first and then returns the updated value.

Question 46. What is the output of this code?

```
var v = 5;
console.log(++v);
console.log(v--);
console.log(v);
```

Output:

```
6
6
5
```

Logical Operators

Question 47. What are logical operators in JavaScript?

The logical operators shown in the code are AND (`&&`), OR (`||`), and NOT (`!`). They combine or reverse conditions.

Question 48. When does the AND (`&&`) expression return `true`?

It returns `true` only when both conditions are true.

Question 49. When does the OR (`||`) expression return `true`?

It returns `true` when at least one of the conditions is true.

Question 50. What does the NOT (`!`) operator do?

It reverses a Boolean result: `!true` is `false`, and `!false` is `true`.

Question 51. What is the output of the following code?

```
var x = 10;
var y = 20;
console.log(x > y && x < y);
console.log(x > y || x < y);
console.log(!(x < y));
```

Output:

```
false
true
false
```

Conditional Statements

Question 52. What is the purpose of an `if` statement?

An `if` statement runs its code block when its condition is true.

Question 53. Why do we use `else if`?

We use `else if` to check another condition when the earlier condition was false.

Question 54. What is the purpose of `else`?

The `else` block runs when none of the preceding `if` or `else if` conditions is true.

Question 55. How can you select a browser using conditional statements?

Store the browser name in a variable and compare it with expected values using `if...else if...else`. The uploaded example checks `Chrome`, `Edge`, and `Firefox`, and prints an invalid-browser message for other values.

Question 56. What is the output when `browserName = "Chrome"` in the uploaded browser-selection code?

```
Chrome browser is selected
```

Question 57. How can you check whether a number is positive, negative, or zero?

Check whether the number is greater than zero. If not, check whether it is less than zero. If neither condition is true, it is zero.

Question 58. What is the output when `inputNumber = 0` in the uploaded positive/negative/zero example?

```
The number is zero
```

Question 59. Why does the zero case belong in the final `else` block?

If the number is neither greater than zero nor less than zero, it must be zero. The `else` block handles that remaining case.

Question 60. What happens if the browser name does not match Chrome, Edge, or Firefox?

The final `else` block executes and prints `Invalid browser name`.

Scenario-Based Questions

Question 61. You have `var a = 10; var b = 3;`. Which operator would you use to find the remainder?

Use the modulus operator (`%`): `a % b` returns `1`.

Question 62. You need to check whether a value is at least 20. Which comparison operator would you use?

Use `>=`, for example `value >= 20`.

Question 63. You want to execute a block only when both conditions are true. Which logical operator should you use?

Use AND (`&&`).

Question 64. You want a condition to be true when either of two conditions is true. Which logical operator should you use?

Use OR (`||`).

Question 65. A browser variable contains `"Safari"`, but the code only checks Chrome, Edge, and Firefox. Which block executes?

The final `else` block executes and prints `Invalid browser name`.